Tlab III: Building the Ridership Dataset (SQL)

Part 1: Overview

In this Tlab, you will step into the role of a Junior Data Analyst at Hudson Mobility Analytics, a (fictitious) consultancy hired by New York City's bike share program. The operations team wants to understand what drives daily ridership but the data to answer that question doesn't exist yet in any single place.

The ride records live in one BigQuery public dataset (over 50 million individual trips). The weather observations live in another (thousands of stations, one table per year). Your job is to use SQL to carve out exactly the data you need, aggregate it to the daily level, join the two sources together, and export a single analysis-ready table. In Part 2, you will use that table to build a model that predicts daily ridership.

This is the single most common pattern in professional data work: ***the modeling dataset doesn't exist until you build it.***

Part 2: Environment Setup

This assignment uses Google BigQuery public datasets, meaning you can complete it entirely in the free BigQuery Sandbox.

Step 1 — Open BigQuery

- Go to https://console.cloud.google.com/bigquery and sign in with your Google account.
- When you land on the page, Google may have already created a project for you automatically (you'll see a project name in the top left corner). That's fine, and you won't need to create a new one.
- If you are prompted to create a project, go ahead and do so. The name isn't terribly important.

Step 2 — Understand the Setup

- Whatever project appears in the top left is yours — it's simply where your queries will run.
- The data for this lab lives in bigquery-public-data, Google's public data library. It is automatically accessible to every BigQuery user, so you don't need to find it, add it, or set it up.

Step 3 — Run Your First Query

- Click the **+ (compose new query)** button to open a query tab.
- Copy and paste the following and hit **Run**:

```
SELECT *
FROM `bigquery-public-data.new_york_citibike.citibike_trips`
LIMIT 10
```

- If you see results, you're ready to go. **Important:** the backticks (`) around the table name are required in BigQuery and are different from single quotes ('). Using single quotes will cause an error.

You will work with three tables in this Tlab:

```
`bigquery-public-data.new_york_citibike.citibike_trips`
`bigquery-public-data.noaa_gsod.gsod20*`
`bigquery-public-data.noaa_gsod.stations`
```

Note the asterisk in gsod20*. NOAA stores each year of weather data in its own table (gsod2013, gsod2014, …). The asterisk is a wildcard that lets one query read many yearly tables at once — you'll use it in Step 7.

Part 3: Schema Reference

Table 1 — citibike_trips (one row per individual bike trip; 50M+ rows). Only the columns you need are listed.

Column Name	Data Type	Description
starttime	TIMESTAMP	When the trip began. Some rows are entirely NULL — real data is messy.
stoptime	TIMESTAMP	When the trip ended
tripduration	INTEGER	Trip length in SECONDS (mind your units)
start_station_name	STRING	Where the trip began (not needed for the final table)

Table 2 — noaa_gsod.gsod20* (one row per weather station per day). Only the columns you need are listed.

Column Name	Data Type	Description
stn	STRING	Weather station ID — you'll find yours in Step 2
year	STRING	Year of the observation. Note: a STRING, not a number
mo	STRING	Month, as a 2-character string (e.g. '07')
da	STRING	Day of month, as a 2-character string
temp	FLOAT	Mean temperature for the day (°F)
max	FLOAT	Max temperature (°F). Reserved word — must be written as `max`
min	FLOAT	Min temperature (°F). Reserved word — must be written as `min`
wdsp	STRING	Mean wind speed (knots). Yes, it's stored as a STRING
prcp	FLOAT	Total precipitation (inches)

Table 3 — noaa_gsod.stations (the directory of every weather station on Earth).

Column Name	Data Type	Description
usaf	STRING	The station ID — matches stn in the weather tables
name	STRING	Station name, in ALL CAPS
country	STRING	Two-letter country code
state	STRING	Two-letter US state code

Part 4: Business Scenario

> **The Scenario:** The operations team believes ridership is driven by weather and by the rhythm of the week, but nobody has ever put the numbers together. Your manager wants one table: one row per day, showing how many rides happened, how long they lasted on average, and what the weather was like. “Get me that table,” she says, “and in Part 2 we'll see if we can actually predict demand.”

Your task: work through the 9 steps below in order. Each step is a query (or an action) that builds toward the final table. Copy each query and a screenshot of its results into your submission document.

Part 5: Guided Steps

Step 1: Build Your Project Repository

Before any SQL, build the home this project will live in, both parts of this Tlab happen inside one GitHub repository. Create a new repository (a name like citibike-ridership-model works; make it public so it can join your portfolio), then set up this exact structure:

```
your-repo-name/
├── code/           eda.ipynb, preprocessing.ipynb, model.ipynb
├── data/           your exported CSV lands here (Step 9)
├── queries/        your final SQL lands here (Step 9)
├── docs/           data dictionary, notes, supporting material
├── README.md
└── .gitignore
```

1. Create the three notebooks in code/ now, as empty files you will fill them in Part 2.
2. Write a skeleton README.md: project title, a one-sentence description, and a short guide to what each folder contains. You will expand it into a full write-up in Part 2.
3. Create a .gitignore containing at least .DS_Store and .ipynb_checkpoints/ these junk files should never appear in a professional repository.
4. Gotcha: git does not track empty folders. Put a placeholder file named .gitkeep inside data/, queries/, and docs/ so they actually push to GitHub.
5. Commit and push. Verify on github.com that the full structure is visible.

Tip: *Graders and later, hiring managers read your repository structure before they read a single line of your code. A clean tree with a real README says “professional” before you've said anything.*

Step 2: Find Your Weather Station

New York has several weather stations, but the operations team wants the one at LaGuardia Airport — major airport stations keep the most complete records. Query the stations table to find every US station whose name contains 'LA GUARDIA', and record its usaf ID. You will use this ID in Step 7.

Tip: *There is a SQL keyword for matching text patterns, and % acts as a wildcard on either side of your search text. Station names are stored in ALL CAPS.*

Step 3: Preview the Ride Data

Before analyzing anything, look at what you're working with. Pull the starttime, stoptime, and tripduration for 10 trips from the citibike_trips table.

Tip: *Never run SELECT * without LIMIT on a table this size. Previewing first is not just politeness — the sandbox gives you a monthly query quota, and careless full-table scans burn through it.*

Step 4: Size Up the Problem

How many trips are in the table, total? Write a query that returns a single number.

In your submission document, answer in one sentence: why can't we just export this table to a CSV and open it in pandas?

Tip: *One aggregate function, no grouping needed.*

Step 5: Rides Per Day

This is the heart of the assignment. Collapse the trip-level data into one row per day: for each calendar date, count the number of rides. Name the date column ride_date and the count num_rides. Exclude the junk rows where starttime is NULL.

Checkpoint: Your dates should run from 2013-07-01 (the month Citi Bike launched) to 2018-05-31 (where this public table ends).

Tip: *starttime is a TIMESTAMP, but you want to group by calendar day. There is a function that extracts just the date part. And there is a specific SQL phrase for keeping only rows where a value is not absent.*

Step 6: Add Average Trip Duration

Extend your Step 5 query: for each day, also compute the average trip length in MINUTES, named avg_duration_min.

Tip: *Check the schema reference for what units tripduration is stored in.*

Step 7: One Station, Six Years of Weather

Now the weather side. From the gsod20* wildcard tables, pull the daily observations for your LaGuardia station for 2013 through 2018. Your query should return: a proper DATE column named obs_date, plus temp, `max`, `min`, wdsp, and prcp — renamed to temp_f, max_temp_f, min_temp_f, wind_speed_knots, and precip_in.

Two puzzles to solve here. First: year, mo, and da are three separate STRING columns, and you need one real DATE. Second: you must limit the wildcard to the right years.

Checkpoint: Exactly 2,191 rows: one per day from 2013-01-01 to 2018-12-31. If you get more, your station filter is wrong; if fewer, your year range is.

Tip: *CONCAT the three string columns with '-' separators, then PARSE_DATE('%Y-%m-%d', ...) the result. Filter the wildcard with _TABLE_SUFFIX BETWEEN '13' AND '18'. And remember the backticks on `max` and `min` — they are reserved words.*

Step 8: The Join — Your Final Query

Assemble the final table. Using WITH, define your Step 6 query as a CTE named daily_rides and your Step 7 query as a CTE named daily_weather. Then INNER JOIN them on the date, and add two more columns computed from ride_date: day_of_week (the day's name, e.g. 'Monday') and month (a number 1–12). Order by ride_date.

Final column list: ride_date, num_rides, avg_duration_min, temp_f, max_temp_f, min_temp_f, wind_speed_knots, precip_in, day_of_week, month.

Checkpoint: Approximately 1,610 rows.
In your submission document, answer: the weather CTE alone had 2,191 rows, but the joined table has ~1,610. Where did the other days go? (Think about what an INNER JOIN keeps, and which of the two sides is missing days.)

Tip: *FORMAT_DATE('%A', …) gives you the weekday name; EXTRACT(MONTH FROM …) gives you the month number.*

Step 9: Export and Commit

Run your final query, then use Save Results → CSV (local file) in the BigQuery console. Name the file citibike_weather_daily.csv.

1. Commit the CSV to the data/ folder of the repository you built in Step 1.
2. Save your final Step 8 query as a file named build_dataset.sql and commit it to a queries/ folder.
3. Push both to GitHub — you will build directly on this CSV in Part 2.

Tip: *Your future self is the customer here. If the CSV is wrong, Part 2 gets much harder — verify the checkpoint row count before you commit.*

SQL Concepts Reference

Every step in this Tlab can be solved using the concepts below. If you are stuck, match the step's goal to the concept that fits.

SQL Concept	What It Does
SELECT ... FROM	The foundation of every query. Pick columns, name the table
LIMIT	Cut off results to the first N rows — essential on huge tables
WHERE / IS NOT NULL	Filter raw rows; keep only rows where a value exists
LIKE '%TEXT%'	Match text patterns; % is a wildcard
COUNT(*) / AVG(col)	Aggregate functions: count rows, average a numeric column
GROUP BY	Bucket rows by a value so aggregates compute per group
AS	Give a column or expression a clean, readable name
DATE(timestamp)	Extract the calendar date from a timestamp
CONCAT(a, b, c)	Glue strings together
PARSE_DATE(fmt, str)	Turn a string like '2016-02-11' into a real DATE
gsod20* / _TABLE_SUFFIX	Query many yearly tables at once; filter which years
WITH name AS (...)	Define a CTE — a named building block for a bigger query
INNER JOIN ... ON	Combine two tables, keeping only rows that match on the key
FORMAT_DATE / EXTRACT	Pull the weekday name or month number out of a date
`backticks`	Required around reserved-word column names like `max` and `min`

Submission Instructions

- Create a new Google Doc titled: **TLAB[N]_SQL_[YourName]**
- For each of Steps 2–8, include: (1) the step number and title, (2) your SQL query, (3) a screenshot of your BigQuery results. Steps 4 and 8 also each have a one-sentence written question, answer them in the doc.

- Complete Steps 1 and 9: your repository must contain the full folder structure from Step 1, data/citibike_weather_daily.csv, and queries/build_dataset.sql.
- Submit the Google Doc link and your GitHub repository link to Canvas before the deadline.